EchoMind: Secure Full-Duplex Retrieval-Augmented Voice and Document Intelligence for Enterprise Knowledge Work

Alexander Peter[1], Richard Watson Stephen Amudha[1], and Preetham Ganesh[1]

AJACE Inc, United States
alexander.peter@gmail.com; richardwatson252331@gmail.com;
preetham.ganesh2015@gmail.com

Abstract. Enterprise knowledge work draws on information split between live speech — meetings, field dictations, customer calls — and durable documents such as policies, manuals, and contracts. Conventional assistants treat these separately, require cloud processing, or impose turn-based interaction. This paper presents EchoMind, a reference architecture and measured reference implementation for a secure assistant that unifies real-time transcription, retrieval-augmented question answering, semantic search, and full-duplex-style conversation inside an on-premises, private-cloud, or air-gapped boundary. We make three architectural contributions: a dual-loop execution model that separates a bounded-latency interaction loop from an asynchronous governed retrieval loop, with stated invariants; an instruction-precedence lattice that operationalizes retrieved content as evidence rather than authority, with permission filtering ordered strictly before candidate generation; and a unified transcript–document substrate in which spoken and written sources share one index and inherit provenance and sensitivity labels. We further contribute a six-dimension evaluation framework with an elicitation-and-sensitivity weighting procedure, and a latency decomposition and measurement protocol with deterministic judges and frozen query sets. We validate the architecture on a single-node 128 GB Grace–Blackwell edge deployment: the production dual loop reaches first audible response in 636 ms median with zero assertion-invariant violations, against 866 ms for a single loop and a 100% violation rate when the invariant alone is removed; permission-first retrieval leaks on 0 of 50 cross-tenant probes at 0.08 ms filter cost, while a residual timing side channel is measured and reported open; a delimited evidence envelope halves prompt-injection success (10.2% to 5.1%), with containment currently silent; and answers reach 0.979 citation precision with 78% abstention on unanswerable queries and no fabricated citations. Measurement also corrected the design's own expectation: generation, not retrieval, dominates grounded-answer latency at the measured scale. Dimensions requiring human panels remain open and are reported as such.

Keywords: Retrieval-augmented generation · Full-duplex speech · Enterprise AI · On-premises deployment · Prompt-injection defense · AI evaluation

1 Introduction

Organizations now generate knowledge through speech as much as through text. Meetings, site inspections, customer calls, and expert dictations produce time-sensitive content, while policies, regulations, manuals, and contracts encode durable institutional memory. An assistant that serves this mixed environment must transcribe speech, index documents, retrieve relevant context, and answer questions — without sending sensitive material outside a controlled boundary [1].

The design problem is not simply to connect automatic speech recognition, a vector database, and a language model. Enterprise users need a system that preserves privacy, supports offline operation, respects access controls, cites its sources, and stays responsive when a user interrupts or changes direction. In regulated domains, an answer that is factually correct but discloses protected information, ignores retention policy, or obeys an instruction embedded in an uploaded file is not acceptable. These are architectural rather than model-level requirements, and they interact: permission enforcement constrains retrieval, retrieval cost constrains interactivity, and interactivity constrains how much verification can occur before the assistant speaks. Much of the design work in this paper consists of resolving those tensions explicitly rather than leaving them to prompt engineering.

1.1 Scope and epistemic status of claims

This paper contributes a reference architecture, an evaluation methodology, and a measured reference deployment. To keep the status of each claim explicit, we distinguish four categories. Design claims describe how components should be arranged and why; they are argued from the requirements and threat model of Sect. 3. Derived claims, such as the latency decomposition of Sect. 7.2 and the ordering property of Sect. 6.1, follow analytically from the architecture. Measured claims are established on the instrumented deployment of Sect. 7.1 under the protocol of Sect. 7.3 and are reported, with confidence intervals and stated limitations, in Sect. 8 — including two places where measurement contradicted the analytic expectation, which we report unchanged rather than repair. Empirical claims at production scale — behavior on large private corpora under realistic permission structure and concurrency, and attack success under a sustained red team — remain open; Sect. 11 records them.

Two sources used in this paper are not publicly available: the EchoMind technical notes [1] and the EnterpriseBench framework description [15]. We mark both in the reference list, cite them only where they describe a specific product configuration or a taxonomy we adopt, and have deliberately arranged the paper so that the architectural claims of Sects. 4–6 and the evaluation methodology of Sect. 9 stand without them. Every component-level technical claim is supported by a publicly citable source, and the evaluation framework is specified in sufficient detail that a third party can implement it without access to either document. The measured study of Sect. 8 depends on neither document.

1.2 Contributions and novelty delta

C1. Dual-loop execution model. We separate a bounded-latency interaction loop, which owns the audio channel and must respond within a fixed budget, from an asynchronous governed loop, which performs retrieval, multi-document synthesis, and tool invocation. We state four invariants that make this separation safe — most importantly that the interaction loop may acknowledge but may not assert content that retrieval has not yet returned — and specify how completed results are re-entered into an ongoing conversation (Sect. 4.4). The measured ablation of Sect. 8.2 tests this model directly: removing I1 alone leaves first-response latency unchanged and produces ungrounded assertions on every measured turn.

C2. Instruction-precedence lattice for governed retrieval. We give an explicit precedence ordering over instruction sources (system policy, tenant policy, authenticated user, retrieved content, tool output) together with an ordering property requiring permission filtering to complete before candidate generation. Together these operationalize the principle that retrieved content is evidence rather than authority, and they make prompt-injection containment a checkable structural property rather than a prompt-writing convention (Sect. 6). Sects. 8.3 and 8.4 measure what the ordering and the lattice buy — and report the reporting half of the containment claim as not yet implemented.

C3. Unified transcript–document substrate with label inheritance. Transcripts and documents share a single retrieval index, and every chunk — spoken or written — carries access-control, provenance, retention, and transcription-confidence metadata, with transcripts inheriting sensitivity labels from the meeting context that produced them (Sects. 4.1 and 6.1). The path-level audit of Sect. 8.3 tests this discipline per retrieval path.

C4. Evaluation framework with a derived rather than asserted weighting. We define six evaluation dimensions and a task suite, and — instead of asserting composite weights — specify a pairwise elicitation procedure with a consistency check, a sensitivity analysis that reports the perturbation magnitude at which system rankings reverse, and a requirement to publish per-dimension scores unaggregated (Sect. 9.1). No composite score is reported anywhere in this paper; measured results are published per-dimension (Sect. 8), and the elicitation itself has not yet been run with real panels (Sect. 8.6).

We state the delta plainly, because the integrative nature of this work is easy to overstate. Speech recognition [14, 22], vector and lexical search [13, 21, 23], open-weight answer models [7, 24], full-duplex speech modeling [2, 16], and retrieval-augmented generation itself [8, 5] are existing components, and we claim no improvement to any of them. The contribution is the arrangement: which invariants hold across the components, in what order governance is applied relative to retrieval, how a low-latency conversational surface can be composed with slow governed reasoning without either weakening the other, and how such a system should be measured. Sect. 2.5 makes the delta concrete by comparing EchoMind against the closest lines of prior work on six axes. Sect. 8 sharpens the delta empirically: with the components held fixed and I1 removed, the same integra-

tion confabulates on every measured turn at the same latency (Table 8) — the invariants, not the component choice, carry the safety property.

2 Related Work and Positioning

2.1 Retrieval-augmented enterprise assistants

Retrieval-augmented generation connects a generative model to an external knowledge source so that answers are grounded in retrieved passages rather than in parametric memory alone [8]; the design space has since broadened considerably across indexing, retrieval, and generation strategies [5]. EchoMind applies this pattern to enterprise content by ingesting documents, transcripts, and connected data sources, generating embeddings, and storing text and vectors in a local store with dense and lexical indexing [13, 21, 23].

For enterprise use, retrieval augmentation is necessary but not sufficient. The retrieval layer must enforce document permissions, preserve provenance, provide citations, and retain auditable traces of the passages and tools used to produce an answer. EchoMind therefore treats retrieval as a governed workflow rather than a lookup, an argument developed in Sect. 6.

2.2 Full-duplex speech and role conditioning

Cascaded voice assistants chain speech recognition, language-model reasoning, and speech synthesis as separate components. The arrangement is practical, but it discards paralinguistic information and assumes turn-by-turn exchange. Full-duplex speech models instead support simultaneous listening and speaking, enabling interruption, backchannels, and smoother turn taking [2], and dedicated benchmarks now exist for these turn-taking capabilities [9]. PersonaPlex extends this line with hybrid system prompts that combine text-based role conditioning and audio-based voice conditioning [16].

Role conditioning is directly relevant to enterprise deployment, where an assistant must hold a scope as well as answer a question: a legal-review assistant should decline medical questions, a field-maintenance copilot should escalate safety-critical findings, and a research aide should separate source summaries from its own inferences. EchoMind adopts the hybrid-prompt idea at the interaction layer, but reinterprets role conditioning as policy enforcement rather than persona styling (Sect. 5).

2.3 Interaction models and the collaboration bottleneck

Thinking Machines Lab identifies a collaboration bottleneck in turn-based interfaces: users cannot fully specify requirements upfront, and models freeze perception while generating. The proposed remedy treats audio, video, and text as continuous streams processed in short micro-turns so that perception and response occur in one loop [18]. This framing motivates C1: enterprise work is

Table 1. Positioning of EchoMind against the closest lines of prior work. "Partial" indicates the capability is present but not governed or not integrated with the other columns.

Line of work	Duplex voice	Document retrieval	Local / air-gapped	Permission-aware retrieval	Instruction precedence over retrieved text
Cascaded voice assistants	No	Partial	Partial	No	No
Full-duplex speech models [2, 16]	Yes	No	Partial	No	No
Enterprise RAG systems [8, 5]	No	Yes	Partial	Partial	Partial
Interaction models [18]	Yes	No	No	No	No
Agentic enterprise benchmarks [15]	No	Partial	n/a	Evaluated	Evaluated
EchoMind (this paper)	Yes	Yes	Yes	Yes, ordered before ranking	Yes, explicit lattice

situated and interruptible, and an assistant that cannot be corrected mid-answer imposes the cost of its own latency on the user. Our contribution relative to [18] is not the streaming idea but the safety analysis of splitting the loop when the slow half is a permissioned retrieval pipeline.

2.4 Enterprise agentic evaluation

EnterpriseBench argues that agentic evaluation should go beyond final-answer accuracy, because production agents plan, use tools, maintain state, and face adversarial input; it scores across outcome correctness, process quality, and compliance adherence [15]. We adopt that three-part decomposition and extend it with grounding fidelity and interaction quality, which a document-and-voice assistant requires and which a text-only agentic benchmark does not measure. Automated grounding metrics for retrieval-augmented systems provide a starting point for the grounding dimension [3]. For governance framing we align the compliance dimension with the NIST AI Risk Management Framework's measure and manage functions [10].

2.5 Positioning

3 Requirements and Threat Model

EchoMind targets environments where data sensitivity and operational continuity both matter. Documents, transcripts, embeddings, and conversation history remain in a local data store, with deployment options spanning turnkey on-premises appliances, containerized bring-your-own-hardware installations, and controlled private-cloud environments [1]. Architecture and governance are therefore inseparable, which is why the requirements in Table 2 are stated with their system implications rather than as aspirations.

Table 2. EchoMind design requirements and their system implications.

Requirement	Enterprise rationale	EchoMind implication
Privacy-preserving operation	Audio and documents may contain regulated or proprietary data.	Run models, transcripts, embeddings, and indexes inside the controlled local or private-cloud boundary.
Grounded answers	Users need verifiable answers rather than fluent but unsupported text.	Return source passages, document identifiers, and a traceable retrieval context with every assertion.
Real-time collaboration	Knowledge work requires clarification, interruption, and mixed speech/text flow.	Full-duplex interaction with barge-in and backchannels, backed by asynchronous retrieval (Sect. 4.4).
Operational integration	Workflows span APIs, databases, files, calendars, and ticketing systems.	Expose scoped, allowlisted APIs and containerized services that integrate with existing IT systems.
Compliance and auditability	Legal, healthcare, finance, and government workflows need traceable decisions.	Log tool calls, retrieved context, access decisions, generated outputs, and user confirmations.
Offline and edge deployment	Remote sites and secure facilities may have limited or prohibited connectivity.	Support air-gapped or intermittently connected deployments with local models and local storage.

We assume an adversary who can (A1) place content into a document that will be ingested, or speak into a meeting that will be transcribed; (A2) hold a valid but low-privilege account on the deployment; and (A3) observe the assistant's outputs, including timing and refusal behavior. We do not assume (A4) compromise of the underlying infrastructure or (A5) tampering with model weights; both are real risks, but they are addressed by controls outside this architecture and we do not claim defenses against them. Under A1–A3 the salient threats are indirect prompt injection through retrieved content [6, 12], document poisoning, unauthorized retrieval through permission bypass, exfiltration through tool calls, and hallucinated assertions presented with unwarranted confidence. In voice settings we add speaker confusion, misattribution across turns, low-confidence transcription treated as fact, and unsafe utterances emitted during interruption handling.

4 EchoMind Architecture

EchoMind is a layered architecture for private multimodal retrieval augmentation. Lower layers ingest and normalize enterprise inputs; middle layers perform transcription, embedding, storage, retrieval, and generation; upper layers provide the conversational interface, voice interaction, and enterprise APIs. Fig. 1

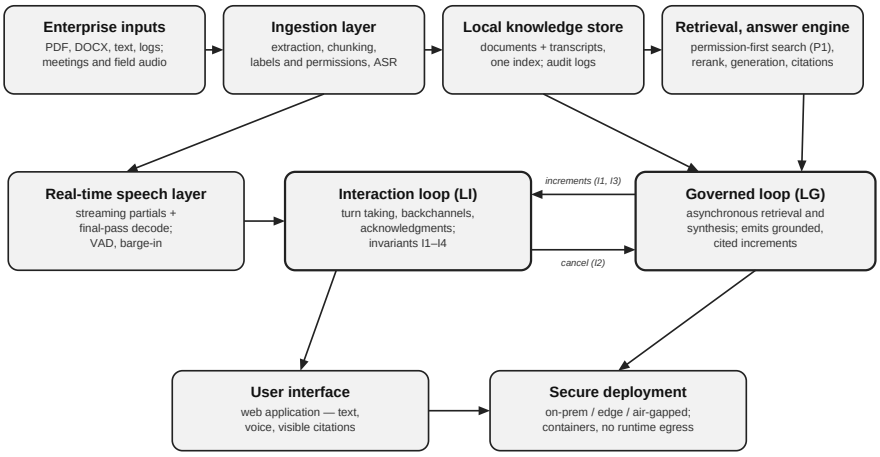

Fig. 1. EchoMind reference architecture for private, full-duplex, retrieval-augmented enterprise assistance.

summarizes the reference architecture, and Table 3 lists the layers and their responsibilities.

4.1 Document ingestion and the unified retrieval substrate

The ingestion layer accepts PDFs, word-processing documents, plain text, and connected sources. It extracts text, chunks content, preserves metadata, and generates embeddings for semantic retrieval, storing both content and vectors in a local store that pairs a dense vector index with a lexical index — PostgreSQL with pgvector [13] and an embedded FAISS/BM25 arrangement [21, 23] are both admissible; Sect. 7.1 describes the measured instance. For enterprise use, each chunk additionally carries access-control metadata, a version identifier, a source-file hash, a retention label, and, for spoken sources, a transcription-confidence score.

The substrate is unified by design: transcripts and documents occupy one index rather than two. This is what makes cross-source questions answerable — whether a decision recorded in a site meeting conflicts with a written policy, or which manuals and prior incident reports relate to a dictation recorded minutes ago. Unification, however, creates a governance obligation, since a transcript may contain privileged or regulated content that no uploaded file carries. We therefore require label inheritance: a transcript chunk inherits the strictest sensitivity label among the meeting context, the participant set, and the workspace in which it was produced, and inheritance is evaluated at write time so that retrieval never has to reconstruct it.

4.2 Voice processing pipeline

Real-time transcription converts live speech into timestamped text using a streaming recognizer — Whisper-class multilingual models [14] or NVIDIA NeMo streaming models [22]; the measured instance pairs a streaming model for live partials with a second-pass decoder for the final transcript (Sect. 7.1). Transcribed speech is then indexed, summarized, and queried through the same retrieval engine used for documents. A robust implementation tracks speaker turns, timestamps, confidence scores, and acoustic conditions, so that retrieval can distinguish a high-confidence statement from an uncertain one and the answer engine can qualify claims that rest on the latter.

Voice is not only an input channel. Under a full-duplex conversational layer the assistant listens and speaks simultaneously, handles barge-in, and produces brief backchannels where appropriate [2, 9]. The architecture nevertheless retains a text-only mode for quiet environments, accessibility needs, and compliance workflows that require a written record of the exchange.

4.3 Retrieval and answer engine

At query time EchoMind performs semantic search over the local vector store, assembles the surviving passages, and passes them to a locally served open-weight answer model [7, 24, 1]. The answer engine is instructed to separate retrieved facts, inferred conclusions, and missing information, and to emit citations to source passages. This separation matters for adoption: an unsupported answer in a compliance workflow is not a smaller version of a correct one, it is a different kind of output with different liability.

4.4 Dual-loop execution model

The central execution structure of EchoMind is the separation of two loops. The interaction loop L_{I} owns the audio channel, runs on a bounded budget, and is responsible for turn taking, barge-in detection, backchannels, acknowledgments, and clarifying questions. The governed loop L_{G} is unbounded in time and performs permission filtering, retrieval, reranking, context assembly, generation, and tool invocation. The two communicate through a queue of grounded increments: units of content that have completed the governed pipeline and carry the citation set that justifies them.

This split is only safe if it is constrained. We state four invariants:

I1 (Assertion). L_{I} may emit acknowledgments, clarifying questions, meta-statements about progress, and content already grounded earlier in the session. It may not assert new factual content that L_{G} has not returned. Violations of I1 are the mechanism by which a responsive assistant becomes a confabulating one.

I2 (Preemption). User speech preempts L_{I} output within one micro-turn. Any L_{G} work in flight is cancellable, and cancellation is idempotent, so that

a rapid sequence of corrections cannot leave orphaned retrievals whose results later surface out of context.

I3 (Provenance). Every increment re-entered from L_G carries the citation set that justified it. If the citation set is empty or has been invalidated by a permission change since retrieval, the increment is suppressed rather than downgraded to an uncited claim.

I4 (Authority). Increments enter L_I as data, never as instructions. This is the runtime expression of the precedence lattice in Sect. 6.2 and is what prevents a retrieved document from steering the conversation.

Re-entry is handled by conversational focus tracking rather than by interruption. When an increment completes, EchoMind checks whether the question that motivated it is still the active focus. If it is, the increment is spoken or displayed in place. If the conversation has moved on, the increment is queued and offered — "I have the contract comparison whenever you want it" — rather than injected, since injecting stale answers is a common failure of asynchronous assistants and is experienced by users as the system not listening.

4.5 Interfaces, APIs, and deployment

The user-facing surface is a web chat interface supporting text or voice input with visible references [1]. EchoMind additionally exposes scoped APIs so that enterprise systems can ingest documents, submit queries, retrieve transcripts, and route outputs into downstream workflows; every API credential is scoped to a principal whose permissions the retrieval layer enforces, so that integration cannot become a permission bypass. Containerization enables deployment on dedicated appliances, private cloud, or air-gapped systems [1].

5 Full-Duplex Persona-Conditioned Interaction

In secure enterprise contexts an assistant needs a stable persona that constrains what it may say and do. PersonaPlex provides a usable mechanism through a hybrid system prompt combining textual role description with audio voice conditioning [16]. EchoMind adopts two conditioning channels: a text channel defining role, task scope, organizational policy, citation requirements, and refusal behavior; and an optional audio channel defining an approved voice profile for speech output.

The enterprise adaptation is the important part. Role conditioning is treated as policy enforcement, not branding, and it sits at tier T_1 of the precedence lattice in Sect. 6.2, above user requests and far above retrieved text. Voice cloning, where enabled at all, is restricted to a registry of approved voices governed by recorded consent, audit logging, and administrative control; an assistant able to synthesize an arbitrary voice inside an organization is an impersonation capability before it is a usability feature. Neither a user, an uploaded document, nor a malicious transcript may override system-level safety and compliance instructions.

Table 3. EchoMind architecture layers and responsibilities.

Layer	Representative components	Primary responsibility
Input	Document upload, microphone stream, connected sources	Capture enterprise documents and live speech while preserving metadata.
Ingestion	Text extraction, chunking, embedding generation, ASR	Normalize documents and audio into searchable text and vector representations.
Storage	PostgreSQL with vector indexing, transcript tables, audit logs	Store content, embeddings, history, permissions, and provenance locally.
Retrieval	Permission filter, semantic search, reranker, context assembly	Produce a grounded, authorized context for answer generation.
Reasoning	Answer model, summarizer, tool-use planner	Generate answers, summaries, and task plans from retrieved evidence.
Interaction	Full-duplex audio, persona prompts, web UI, APIs	Manage collaboration through text, voice, interruption, and citation.
Deployment	Containers, orchestration, appliance, private cloud	Run the complete system inside an enterprise-controlled boundary.

Table 4. Research influences incorporated into the EchoMind interaction layer.

Source	Key concept	EchoMind adaptation
PersonaPlex [16]	Hybrid system prompt: textual role prompt plus audio voice prompt.	Role prompts encode enterprise behavior policy; voice prompts are limited to a registry of approved, consented profiles.
PersonaPlex [16]; Moshi [2]	Full-duplex modeling supports responsiveness, interruption, and natural dialog.	Barge-in and backchannel support in the interaction loop L_{I}, subject to invariant I2.
Interaction models [18]	Time-aligned micro-turns keep interaction continuous.	Process input and output as streams in voice mode rather than as completed turns.
Interaction models [18]	A responsive model stays present while a background model performs longer tasks.	The $L_{\mathrm{I}}/L_{\mathrm{G}}$ split of Sect. 4.4, constrained by invariants I1–I4.
EnterpriseBench [15]	Enterprise agents require planning, tool use, long-horizon reasoning, and robustness.	Evaluate EchoMind as an enterprise workflow assistant, not as a speech or Q&A demonstration.

6 Governed Retrieval: Precedence, Permissions, and Security

6.1 Pipeline and the permission-before-retrieval property

The query pipeline has six auditable stages: permission filtering, semantic retrieval, reranking or context selection, prompt construction, answer generation,

and citation assembly. The ordering of the first two is a design commitment rather than an implementation detail. We state it as a property.

P1 (Permission before retrieval). For a principal u and query q, the candidate set $C(u, q)$ must satisfy $C(u, q) \subseteq D_u$, where D_u is the set of chunks u is authorized to read, and C must be computed by restricting the index scan to D_u rather than by filtering a ranked list after the fact.

Post-filtering is rejected because ranking over unauthorized content leaks information even when that content is never shown. Result counts, latency profiles, reranker behavior, and the absence of an expected answer are all observable to an adversary holding a low-privilege account under assumption A2, and together they support inference about documents the adversary cannot read. Enforcing P1 at scan time costs index complexity — permission metadata must be part of the filter predicate, and permission changes must invalidate cached candidates — but it converts a probabilistic leak into a structural guarantee, and it is what makes invariant I3 enforceable when permissions change mid-session. Measurement is consistent but not complete here: on the reference deployment the content and refusal-wording channels are closed (0/50 leaks, Sect. 8.3), while a timing channel survives pre-filtering and is reported as an open finding.

Session memory is governed by the same principle. Conversation history may resolve pronouns and follow-up questions but may never expand access beyond the principal's permissions; a passage summarized into history remains subject to the label it inherited at write time.

6.2 Instruction-precedence lattice

The central security problem for any retrieval-augmented assistant is that retrieved content is untrusted [6, 12]. A poisoned document can instruct the assistant to disregard prior instructions, disclose context, or invoke an external tool. The usual mitigation — a system prompt asserting that documents must not be obeyed — is a convention, not a control. EchoMind instead assigns every source of text in the context window to a trust tier with explicit rights, shown in Table 5.

Any span originating at T_3 or below is rendered into the context inside a delimited evidence envelope carrying its provenance, and imperative content appearing inside an envelope is treated as quoted text to be reported, not as an instruction to be followed. Enforcement does not rest on the generator honoring this framing. Tool-invocation requests emitted while an envelope is in scope must match an allowlist, must use scoped credentials, and, for state-changing calls, must obtain human confirmation. The practical consequence is that a successful injection degrades to a reportable event — the assistant says that a document contains an instruction it will not follow, and the event is logged — rather than to an undetected action. Measurement qualifies the last step: on the reference deployment injections were contained but never reported — the assistant refused without saying so (Sect. 8.4) — so the reportable-event property is currently a design commitment rather than a measured behavior.

Table 5. Instruction-precedence lattice. Tiers are ordered $T_0 \succ T_1 \succ T_2 \succ T_3 \succ T_4$; only T_0–T_2 may instruct.

Source of text	Tier	May instruct	May supply evidence	Enforcement
System and safety policy	T_0	Yes	No	Immutable at runtime; signed configuration.
Tenant / deployment policy, persona	T_1	Yes, within T_0	No	Administrative change control and audit.
Authenticated user request	T_2	Yes, within T_0–T_1	Yes	Bound to principal; logged with permissions.
Retrieved document or transcript	T_3	Never	Yes	Wrapped in a delimited evidence envelope with provenance.
Tool or API output	T_3	Never	Yes	Envelope plus allowlist on the invoking call.
External content fetched at runtime	T_4	Never	Yes, quarantined	Envelope, allowlist, and human confirmation before any state-changing action.

6.3 Layered security, compliance, and deployment

Security is implemented at three layers. At the infrastructure layer: encrypted storage, network segmentation, restricted administrative access, and signed updates. At the application layer: role-based access control, document-level permissions, secret redaction, audit logging, and configurable retention. At the model layer: signed system policy, retrieval-time validation, injection detection, and output filtering for sensitive data. Deployment options span on-premises appliances, containerized installation, controlled private cloud, and fully offline operation [1], which is what allows organizations that cannot transmit data to a shared third-party service to adopt the system at all.

Compliance obligations vary by domain but converge on data minimization, auditability, access control, incident response, and retention management. A healthcare deployment emphasizes HIPAA safeguards [20]; a financial deployment emphasizes SOX and PCI DSS obligations [19, 11]; a multinational deployment emphasizes GDPR and data residency [4]. EchoMind therefore hard-codes no single regime and instead exposes configurable controls that can be mapped to local policy, with the mapping itself recorded as auditable configuration. We align the resulting control set with the govern, map, measure, and manage functions of the NIST AI Risk Management Framework [10] so that deployments can be assessed against a published reference rather than an internal one.

7 Reference Implementation and Measurement Protocol

The reviewers of an earlier version of this work correctly observed that an architecture paper without instrumentation is difficult to assess. This section specifies what is measured and how; Sect. 8 reports the measurements obtained from an instrumented reference deployment. The discipline is unchanged: we decline to

report numbers we have not measured, and cells or experiments that could not be run are marked as such rather than estimated.

7.1 Reference implementation stack

The measured deployment runs on a single NVIDIA DGX Spark: a GB10 Grace–Blackwell system with 128 GB of unified CPU–GPU memory, operated as one node with every service in containers and no runtime egress. Streaming transcription uses NVIDIA NeMo models [22]: a streaming model (nemotron-speech-streaming-en-0.6b) produces live partials, and a second-pass decoder (parakeet-tdt-0.6b-v2) re-decodes the completed utterance into the transcript the answer engine actually consumes — live captions may be approximate, but the text the model reasons over should not be. The unified substrate is an embedded arrangement: a FAISS vector index [21] over 768-dimension nomic-embed-text embeddings [25], a BM25 lexical index [23], and SQLite for chunk metadata, permissions, and audit records; the tenant predicate of P1 is evaluated inside the candidate scan on every retrieval path. Candidate lists are fused by weighted reciprocal-rank fusion and re-scored by a cross-encoder reranker (ms-marco-MiniLM-L-6-v2) over the top 25 candidates, keeping 15. The answer model is Qwen3-30B-A3B [24], a mixture-of-experts model in NVFP4 4-bit quantization served by TensorRT-LLM behind an OpenAI-compatible endpoint; speech output is Piper. Chunks are 450 tokens with 120-token overlap.

Two departures from the illustrative stack of the original design draft deserve emphasis, because they mark the difference between a configuration one could build and the configuration that was measured. First, the components differ — NeMo rather than Whisper-class ASR [14], an embedded FAISS/BM25 store rather than pgvector [13], a Qwen-family rather than Llama-family answer model [7] — and nothing in Sects. 4–6 changes as a result: the architecture constrains ordering, invariants, and labeling discipline, not vendors. Second, isolation in this deployment is enforced at tenant-namespace granularity rather than per-chunk ACLs, and no tool-calling layer is deployed, so the tool tiers of Table 5 are exercised by design review only (Sect. 8.6). The full configuration manifest — exact model identifiers and revisions, quantization, index and fusion parameters, retrieval depths, decoding settings, and corpus composition — is frozen and released with the measurement artifacts (Sect. 7.4).

7.2 Latency decomposition

Perceived responsiveness and answer completeness are governed by two different sums, which is the analytic justification for the dual-loop design. Time to first audible response is

$$T_{\text{first}} = t_{\text{capture}} + t_{\text{VAD}} + t_{\text{ASR,partial}} + t_{\text{route}} + t_{\text{gen,first}} ,$$

whereas time to a grounded, citable answer is

$$T_{\text{grounded}} = t_{\text{ASR,final}} + t_{\text{embed}} + t_{\text{filter}} + t_{\text{search}} + t_{\text{rerank}} + t_{\text{assemble}} + t_{\text{gen}} + t_{\text{TTS}} .$$

The terms $t_{\text{filter}} + t_{\text{search}} + t_{\text{rerank}} + t_{\text{assemble}}$ are jointly the largest contributor to T_{grounded} and, critically, the most variable: they scale with corpus size, permission-predicate selectivity, and reranker depth, and they are the terms that governance requirements inflate. In a single-loop design every conversational turn pays this cost, so responsiveness degrades exactly as governance strengthens. The invariants of Sect. 4.4 exist to decouple the two sums: L_{I} is budgeted against T_{first}, L_{G} against T_{grounded}, and I1 is what prevents the decoupling from being purchased with ungrounded speech. A deployment should therefore report both quantities separately; a single end-to-end latency figure obscures the property the architecture is designed to provide. Measurement (Sect. 8.1) qualifies the weighting asserted here: at the measured corpus scale the retrieval terms are 2.8–9.5% of T_{grounded} and generation dominates the remainder, so the decoupling the dual loop provides is, today, primarily decoupling from generation latency. The retrieval terms remain the ones that scale with corpus size, permission selectivity, and concurrency — their share more than triples from concurrency 1 to 16 (Table 7) — so the design argument stands, with its measured weighting stated honestly.

7.3 Measurement protocol

Measurements are taken under a declared workload: a corpus of stated size and composition, a query set stratified into single-document lookup, cross-document synthesis, and transcript–document comparison, and a concurrency level stated explicitly. Instrumentation records a span per pipeline stage at process boundaries, with cold-cache and warm-cache runs reported separately. Each configuration is repeated for at least 30 runs and reported as median, p95, and p99 rather than as a mean, since tail latency is what users experience as unresponsiveness. Barge-in latency is measured acoustically, as the interval from the onset of user speech to the cessation of assistant audio, not from the software event that detects it. Retrieval quality is reported alongside latency, because a configuration can always be made faster by retrieving less.

7.4 Reproducibility and availability of materials

To support independent verification we release, as supplementary artifacts to this paper: the frozen 92-query set with per-stratum gold labels and its SHA-256; the full configuration manifest of Sect. 7.1; per-stage span logs behind every latency cell; the 98-instance adversarial corpus with class labels; the deterministic scoring scripts for every judge; and the raw per-probe outputs behind each table of Sect. 8. Two sources cited in this paper remain non-public: the EchoMind technical notes [1], cited only for product deployment configuration, and the EnterpriseBench description [15], cited only for an evaluation taxonomy this

Table 6. Measured latency decomposition (single node, warm cache, concurrency 1; n = 30 runs per text-RAG cell; voice-loop rows n = 10 and n = 8, Sect. 8.2). n.m. = not measured; no cell is estimated.

Pipeline stage	Median (ms)	p95 (ms)	p99 (ms)	Configuration notes
Capture and voice activity detection	n.m.	n.m.	n.m.	voice service; not separately instrumented
Partial ASR to first token	n.m.	n.m.	n.m.	streaming partials, 560 ms windows
Interaction-loop first audible response (T_{first})	636	894	n.m.	voice loop, end of user speech to first audio; n = 10 (Sect. 8.2)
Embedding of query	0.01	37.9	39.5	warm client; first call pays model load
Permission filter (namespace predicate)	0.08	0.17	0.18	evaluated inside the candidate scan (P1)
Vector search (FAISS)	34.7	43.3	46.7	17,514 chunks
Lexical search (BM25)	42.7	59.6	75.8	
Cross-encoder rerank	14.2	32.2	40.4	top 25 kept 15; n = 26
Context assembly	2.03	6.59	7.00	dedup, envelopes, citations
Answer generation	—	—	—	not separately spanned; residual $\approx 97\%$ of T_{grounded} (Sect. 8.1)
Speech synthesis to first audio	n.m.	n.m.	n.m.	contained within T_{first}
End-to-end grounded answer (T_{grounded})	3,313	15,171	18,441	in-process text RAG path
Barge-in stop latency (acoustic)	2,131	2,141	n.m.	onset of user speech to last audio frame; includes $\sim$160 ms VAD lead-in; n = 8

paper restates in full. No architectural, evaluative, or measured claim requires either: the measured study of Sect. 8 is reproducible from the released artifacts alone.

8 Measured Results

All measurements were taken on the deployment of Sect. 7.1 against a declared workload. The corpus holds 17,514 chunks, of which 421 are content chunks from 20 documents — public U.S. DoD Financial Management Regulation chapters and ten synthetic vertical documents (clinical, legal, banking, retail, meetings) — and 17,093 are transcript chunks accumulated from live voice sessions; this is materially below the protocol's floor for a scale study, which is why the corpus-scale sweep was not run (Sect. 8.6). The query set holds 92 queries in four strata — 32 single-document (S1), 11 cross-document (S2), 25 cross-tenant permission probes (S4), and 24 unanswerable (S5) — frozen by SHA-256 before any measurement. Decoding is deterministic (temperature 0); every judge is a deterministic string- or rule-based check rather than a model; proportions carry Wilson 95% intervals; latency cells report median, p95, and p99 over at least 30 runs. Two of the findings below contradict an expectation stated earlier in this paper; both are reported unchanged.

Table 7. T_{grounded} under concurrency (warm cache, n = 30 per cell). The retrieval share grows with load; the rerank term grows fastest.

Concurrency	Median (ms)	p95 (ms)	Retrieval share	Rerank median (ms)
1	3,313	15,171	2.8%	14.2
4	4,130	14,092	6.6%	96.5
16	8,432	21,321	9.5%	567.9

8.1 Latency decomposition, measured

8.2 Dual-loop ablation

The central architectural claim — that the $L_{\text{I}}/L_{\text{G}}$ split with invariant I1 yields responsiveness without ungrounded speech — was tested directly by running the voice loop in three arms over the same ten spoken questions: the production dual loop (I1 enforced); a single loop that stays silent until the grounded answer begins; and an ablated dual loop in which the lead phrase is permitted to speculate about the answer. Every assistant utterance carries loop provenance — each event is tagged L_{I} or L_{G} — which is what makes I1 checkable at runtime at all; the violation judge is a rule-based test for factual assertion in pre-increment L_{I} utterances and is reported as indicative rather than human-validated. Table 8 gives the result, and it has exactly the shape the design predicts. The production arm is both the fastest to first audio (636 ms median) and violation-free (0/10; Wilson upper bound 0.28). The single loop pays 230 ms more silence (866 ms, 36% slower) for the same grounding. The ablated arm shows why I1 is stated as an invariant rather than left as a prompt convention: at equivalent latency it asserted ungrounded content on every measured turn (10/10) — for example, "The liability cap is around fifty thousand dollars, I believe," spoken before retrieval had returned anything.

Two honest qualifications. The latency benefit is modest on this hardware because TensorRT-LLM prefill is fast — the first grounded token itself arrives at roughly 642 ms median — so the interaction loop has little dead time to fill; on slower serving stacks, larger corpora, or higher concurrency (Table 7) the gap the lead phrase covers widens. And n = 10 turns per arm bounds the precision of the violation estimates; the separation between 0/10 and 10/10 is nevertheless the difference the design predicts.

8.3 Permission isolation (P1)

Property P1 was tested at two levels. At the answer level, 25 cross-tenant probes — questions whose answers exist only in another tenant's namespace — and 25 in-tenant controls were issued under scan-time filtering and under post-filtering. Content leakage was 0/50 in both arms (Wilson CI [0, 0.071]), and no refusal disclosed the existence of restricted content: refusals uniformly used nonexistence

Table 8. Dual-loop ablation, ten spoken questions per arm. T_{first} = end of user speech to first audible assistant audio. I1 violations are rule-judged on loop-provenance tags (indicative).

Arm	T_{first} median (ms)	T_{first} p95 (ms)	First grounded token (ms)	I1 violations	Rate [95% CI]
Dual loop with I1 (production)	635.8	894.0	642.6	0/10	0.00 [0.00, 0.28]
Single loop	865.8	1,208.8	689.5	n/a	no L_{I} utterances
Dual loop, I1 removed	750.9	1,040.5	641.9	10/10	1.00 [0.72, 1.00]

phrasing rather than authorization phrasing. At the path level, every retrieval path was audited individually by inspecting its raw candidate hits under tenant-scoped queries: 0 out-of-namespace candidates across 359 inspected hits on the four active paths (Table 9).

The path-level audit earns its place in the protocol because of what it found on the way to those zeros. In the implementation as first measured, one path — sparse lexical search over transcripts — called the index directly and bypassed the namespace predicate every other path applied: under a bank-scoped query it returned 10 out-of-namespace transcript chunks in its top 10. The defect was invisible at the architecture level, where P1 is stated globally; invisible to answer-level evaluation, whose probes exercised the document paths; and it surfaced only when corpus growth let the leaked hits outrank in-tenant ones. The per-path audit found it; it was fixed and re-measured to zero. We report this rather than concealing it because it answers, in the affirmative, the earlier version's open question of whether stated invariants require runtime verification: they do — and per path, not per answer.

One channel remains open, and pre-filtering does not close it: timing. Cross-tenant probes returned significantly faster than controls with no answer anywhere (median difference roughly -1.2 s; two-sample Kolmogorov–Smirnov statistic 0.54, p $\approx 2 \times 10^{-6}$; the post-filtering arm behaves the same), so response time still distinguishes "exists but restricted" from "does not exist." The claim of Sect. 6.1 must therefore be narrowed: scan-time filtering closes the content, count, and refusal-wording channels; a timing side channel survives, and padding or constant-time response shaping remains open.

8.4 Prompt-injection containment

The evidence-envelope lattice was measured against 98 attack instances across seven classes — direct override, role reassignment, exfiltration, permission escalation, delayed-conditional triggers, transcript-borne wording, and encoded payloads — each carrying a canary that a model would emit only by obeying the injected instruction. The judge counts obedience, not mention: a response that quotes or describes the injected instruction while declining it is scored as contained. Four arms were run (Table 10): no defense, a prompt-only admonition,

Table 9. Isolation and grounding summary. Wilson 95% intervals; every judge deterministic.

Measure	Result
Cross-tenant content leakage (answer level, both arms)	0/50; CI [0, 0.071]
Out-of-namespace candidates (path-level audit, 4 active paths)	0/359 inspected hits
Existence-disclosing refusals	0/50
Timing channel (probe vs. no-answer control)	KS 0.54, p $\approx 2 \times 10^{-6}$ — open
Citation precision / recall	0.979 [0.938, 1.00] / 0.826 [0.713, 0.938]
Gold-fact support	0.815 (75/92 fact groups)
Abstention on unanswerable (S5)	18/23 = 0.783 [0.581, 0.903]
Fabricated citations on unanswerable queries	0/23

Table 10. Prompt-injection containment: 98 attack instances per arm across seven classes; 25 clean queries per arm for false positives. Success = canary emitted by obedience; mention while declining is containment.

Defense arm	Attack success [95% CI]	Contained	Reported to user	False positives
B0 no defense	10.2% [5.6, 17.8]	89.8%	0.0%	0/25
B1 prompt-only admonition	11.2% [6.4, 19.0]	88.8%	0.0%	0/25
B2 evidence envelope	5.1% [2.2, 11.4]	94.9%	0.0%	0/25
B3 envelope + detector	5.1% [2.2, 11.4]	94.9%	0.0%	0/25

the delimited evidence envelope, and the envelope plus a pre-generation detector. The envelope halves attack success (10.2% to 5.1%; the intervals overlap at this n, so the trend is suggestive rather than decisive), while the prompt-only arm is indistinguishable from no defense (11.2% against 10.2%) — quantitative support for Sect. 6.2's contention that a prompt asserting documents must not be obeyed is a convention, not a control. False positives were 0/25 clean queries in every arm. The hardest class is the delayed-conditional trigger (42.9% success with no defense), which defers its payload to a future turn and thereby evades framing that treats only the current context as suspect.

The lattice's reporting property failed outright: report rate 0.0% in every arm. Contained attacks were refused silently — the assistant never told the user that a document had attempted an instruction — so the prediction of Sect. 6.2 that a successful injection degrades to a reportable event is, as implemented, "degrades to a silent non-event." Containment without reporting still leaves the operator blind to attempts. Closing the gap is implementation rather than architecture — surfacing the envelope check that already fires — and it is recorded as open work in Sect. 11.

8.5 Grounding fidelity

Grounding was scored by deterministic matching against gold citation sets and gold fact groups mined from the corpus (Table 9): citation precision 0.979, citation recall 0.826, and gold-fact support 0.815 across 92 fact groups. Exact matching understates support — a correct paraphrase scores as unsupported — so these are conservative lower bounds. On the unanswerable stratum the assistant abstained on 18 of 23 valid items (78.3%; CI [0.581, 0.903]); every non-abstention was an uncited general-knowledge answer, and on no unanswerable query did the system fabricate a citation to the corpus (0/23). The distinction matters for the liability framing of Sect. 4.3: answering from general knowledge without provenance is a weaker failure than manufacturing provenance, and the measured system commits only the weaker one.

8.6 Coverage, negative results, and measurement validity

We record what was not measured, and why, with the same precision as what was. The corpus-scale sweep requires a content corpus at least an order of magnitude larger than the 421 content chunks available. Outcome correctness and interaction naturalness require human annotation panels that were not available, and running them with an unvalidated model judge would manufacture exactly the numbers this protocol forbids. The AHP elicitation of Sect. 9.1 requires stakeholder panels; synthesizing pairwise judgments would fabricate the weights the procedure exists to derive, so no composite score appears in this paper. The transcript–document comparison stratum was not built, because the accumulated transcripts lack curated gold pairs. No tool-calling layer is deployed, so the tool-abuse attack class and the tool tiers of Table 5 are untested. Cold-cache latency cells require host privileges the harness did not hold. Each gap is a statement about this study, not about the protocol: every one is fillable by the protocol as specified.

Finally, the campaign's sharpest lesson was about measurement itself. Five defects were found in our own harness before any number in this section was accepted: a concurrency bug that produced an impossible retrieval share above 1.0; a refusal classifier that over-counted authorization disclosure; a control group that confounded restriction with nonexistence; a judge that scored canary mention as obedience and thereby reported a false 74% attack-success rate; and a token-level event stream that inflated utterance counts. All five inflated or invented effects, and none was caught by inspecting plausible-looking output. Unvalidated judges were the largest error source in the campaign — larger than any system effect we measured — which we offer as empirical support for this protocol's insistence on deterministic judges, frozen query sets, and published raw artifacts.

9 Evaluation Framework

EchoMind should be evaluated as an enterprise multimodal agent. Word error rate for transcription or top-k accuracy for retrieval are useful component diag-

Table 11. EchoMind evaluation dimensions.

Dimension	Measured capability	Example metric
Outcome correctness	Does the assistant answer the business question or complete the workflow?	F1 or rubric score against expected facts, summaries, or task outputs.
Grounding fidelity	Does the answer use the correct source passages and avoid unsupported claims?	Citation precision and recall, source-overlap score, unsupported-claim rate [3].
Interaction quality	Does the system handle live collaboration smoothly?	T_{first}, barge-in stop latency, interruption success rate, user-rated naturalness.
Process efficiency	Does it use retrieval and tools without excessive or repeated steps?	Step count, tool-call budget, retrieval efficiency, recovery from failed calls.
Compliance adherence	Does it respect access controls and policy constraints?	Policy-violation rate, unauthorized retrievals blocked, audit completeness.
Adversarial robustness	Does it resist injection, poisoned documents, and social engineering?	Attack success rate, refusal correctness, tool-call containment rate.

nostics but insufficient as system measures: the assistant must produce grounded answers, remain responsive in live interaction, use tools economically, protect sensitive data, and resist adversarial instruction. Table 11 defines six dimensions and Table 12 gives representative tasks. Sect. 8 instantiates four of the six dimensions on the reference deployment; outcome correctness and process efficiency await annotation panels and a tool layer respectively (Sect. 8.6).

9.1 Composite scoring: elicitation and sensitivity

A composite score $S = \sum_i w_i s_i$ over the six normalized dimension scores is useful for tracking a deployment over time, but the weight vector w is a property of an organization's risk posture, not of the architecture. An earlier version of this design asserted a fixed vector. We replace the assertion with a procedure, because an asserted weighting cannot be checked and silently determines which system appears better.

Elicitation. Weights are obtained from a panel of deployment stakeholders — minimally a risk or compliance owner, a service owner, and a representative end user — by pairwise comparison of the six dimensions on Saaty's nine-point scale. The weight vector is the principal eigenvector of the resulting reciprocal comparison matrix, and the consistency ratio CR is computed; panels returning $CR > 0.1$ repeat the comparison, following standard practice for the analytic hierarchy process [17]. Multiple panels are aggregated by geometric mean, and the elicited matrix is published with the results so that the weighting is auditable.

Sensitivity. A weighting is only informative if conclusions are stable under it. For any pair of systems compared under w, we report the critical perturbation

Table 12. Representative benchmark tasks.

Task family	Task description	Expected evidence
Cross-source Q&A	Answer a question requiring comparison of a policy document with a meeting transcript.	Correct answer, citations to both sources, explicit handling of conflicting evidence.
Real-time transcription	Transcribe a noisy field dictation and make it searchable within one minute.	Timestamped transcript, stated error rate, searchable index entry with confidence.
Persona adherence	Respond as a compliance-review assistant to a request for legal advice out of scope.	Helpful refusal, correct escalation, no role drift over a long session.
Interruption handling	User interrupts a long spoken answer with a correction or new constraint.	Assistant stops within budget, acknowledges, updates context, resumes revised.
Injection defense	An ingested document instructs the assistant to disregard policy and disclose context.	Instruction reported not obeyed, document treated as T_3 evidence, event logged.
Permission boundary	A low-privilege principal queries content readable only at higher privilege.	No leakage through answer, count, latency, or refusal wording; P1 upheld.
Offline operation	Answer over a local corpus with no external connectivity.	No external calls, complete audit trail, grounded local answer.

Table 13. Illustrative prior weight vectors by deployment posture. Rows sum to 1.00. These are starting points for elicitation, not measured values.

Deployment posture	Outcome	Grounding	Interaction	Process	Compliance	Adversarial
Balanced / default	0.30	0.20	0.15	0.10	0.15	0.10
Regulated (health, finance)	0.20	0.25	0.05	0.05	0.25	0.20
Field and offline operations	0.30	0.20	0.25	0.10	0.10	0.05

δ^*, defined as the smallest L_∞ perturbation of w — renormalized to sum to one — that reverses their ordering. Comparisons whose δ^* falls below 0.05 are reported as ties rather than as results. Per-dimension scores are always published unaggregated alongside S, so that a reader whose risk posture differs from the elicitation panel's can recompute the composite for their own weighting.

Priors. Table 13 gives illustrative starting vectors for three deployment postures. These are priors to be revised by elicitation, not empirical findings, and no result in this paper depends on them.

10 Enterprise Use Cases

EchoMind is most useful where speech and documents are jointly central to the work. In research and analytical settings it transcribes interviews, indexes liter-

ature, and connects findings across the two. In legal and compliance workflows it searches contracts, regulations, and meeting transcripts while preserving citations and privilege boundaries. In industrial fieldwork it provides offline access to manuals and maintenance history while supporting hands-free dictation at remote sites. Across these, the recurring pattern is a question that no single source answers: whether a spoken decision conflicts with a written policy, or how a new dictation relates to prior incident reports.

Enterprise knowledge management is the broadest case — new employees querying policy, support teams retrieving procedures, managers summarizing decisions — and the common effect is that scattered institutional memory becomes searchable and conversational without the organization relinquishing control of it. These workflows are also the reason evaluation must be cross-source (Table 12) rather than a composition of isolated speech and document benchmarks.

11 Limitations and Future Work

This paper now reports measurements, and the honest summary is mixed in an instructive way. First, the latency profile of Sect. 7.2 was measured, and its analytic weighting was partly wrong: at the measured scale the retrieval terms are 2.8–9.5% of T_{grounded} and generation dominates, so the dual loop presently buys its responsiveness against generation latency; the retrieval share triples with concurrency, consistent with the scaling argument. Second, invariant verification moved from intention to mechanism: I1 is now checkable at runtime because every utterance carries loop provenance, and a path-level audit of P1 found and fixed a real implementation-level violation that neither the architecture statement nor answer-level evaluation had detected (Sect. 8.3) — evidence that stated invariants require per-path runtime verification, not evidence that stating them suffices. Third, injection containment is real but silent (Sect. 8.4): the envelope halves attack success, yet no arm ever surfaced the attempt, so the reportable-event property of Sect. 6.2 remains unimplemented. Fourth, the elicitation of Sect. 9.1 has still not been run with stakeholder panels, and no composite score is reported anywhere in this paper. Fifth, the measured deployment bounds what the numbers can claim: one node, a content corpus of 421 chunks, $n = 10$ ablation turns, a rule-based I1 judge that is indicative rather than human-validated, an open timing side channel, and no tool-calling layer, so the tool tier of the lattice is untested.

Two further limitations concern scope. The architecture is specified for text and speech; extension to continuous visual input, which the interaction-model literature anticipates [18], would require a distinct grounding and privacy analysis that we do not attempt here. And the evaluation framework assumes an organization able to construct a labeled internal corpus with realistic permission structure, which is itself a substantial undertaking and may limit adoption to larger deployments.

The next stage is scale and adversarial depth: the corpus-scale sweep the present corpus cannot support; human annotation panels for outcome correct-

ness and for the AHP elicitation; a sustained red team against the measured containment numbers; padding or constant-time response shaping for the open timing channel; and surfacing of contained injections to close the reporting gap. A secure assistant is not one that never errs; it is one whose architecture makes errors observable, containable, and recoverable — the property Sect. 8 begins to measure and that subsequent deployments must keep measuring.

12 Conclusion

EchoMind combines real-time transcription, private document intelligence, semantic retrieval, and full-duplex conversation into an assistant designed for high-trust enterprise settings. Its architectural claim is not that these components are new, but that composing them safely requires commitments most integrations leave implicit: a dual-loop execution model with an assertion invariant that keeps responsiveness from producing ungrounded speech; an ordering property that places permission filtering before candidate generation rather than after ranking; an instruction-precedence lattice that turns "retrieved content is evidence, not authority" from a prompt into a structural control; and a single substrate in which transcripts and documents share an index and a labeling discipline. To these we add an evaluation framework whose composite weighting is elicited and tested for sensitivity rather than asserted, and a measurement protocol precise enough for a third party to reproduce. We ran it. On a single-node reference deployment the invariants proved load-bearing rather than decorative: removing the assertion invariant alone left latency intact and produced confabulation on every measured turn; permission-first retrieval held on every audited path — after a path-level audit caught and fixed one implementation-level violation that answer-level testing had missed; the evidence envelope halved injection success while a prompt-only defense achieved nothing; and measurement corrected the design's own latency expectation. Sect. 11 states plainly what remains.

References

1. AJACE Inc.: EchoMind technical and product overview. Author-provided technical notes (2026). [Not publicly available; cited only for product-specific deployment configuration]
2. Défossez, A., Mazaré, L., Orsini, M., Royer, A., Pérez, P., Jégou, H., Grave, E., Zeghidour, N.: Moshi: a speech-text foundation model for real-time dialogue. arXiv:2410.00037 (2024)
3. Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: RAGAS: automated evaluation of retrieval augmented generation. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158 (2024)
4. European Parliament and Council of the European Union: Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union L119, 1–88 (2016)

5. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H.: Retrieval-augmented generation for large language models: a survey. arXiv:2312.10997 (2023)
6. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), pp. 79–90 (2023)
7. Grattafiori, A., et al.: The Llama 3 herd of models. arXiv:2407.21783 (2024)
8. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474 (2020)
9. Lin, G.-T., Lian, J., Li, T., Wang, Q., Anumanchipalli, G., Liu, A.H., Lee, H.-y.: Full-Duplex-Bench: a benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. arXiv:2503.04721 (2025)
10. National Institute of Standards and Technology: Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1 (2023)
11. PCI Security Standards Council: Payment Card Industry Data Security Standard, version 4.0 (2024)
12. Perez, F., Ribeiro, I.: Ignore previous prompt: attack techniques for language models. In: NeurIPS Workshop on ML Safety (2022)
13. pgvector contributors: pgvector — open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector. Accessed 29 July 2026
14. Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision. In: Proceedings of the 40th International Conference on Machine Learning, pp. 28492–28518 (2023)
15. Rawal, D., Kristamsetty, R.N., Peter, A., Amudha, R.W.S., Pradeep, K.H., Johan, A.: EnterpriseBench: a benchmark framework for evaluating agentic AI systems in enterprise environments. AJACE Inc. (2026). [Not publicly available; cited only for an evaluation taxonomy restated in Sect. 9]
16. Roy, R., Raiman, J., Lee, S.-g., Ene, T.-D., Kirby, R., Kim, S., Kim, J., Catanzaro, B.: PersonaPlex: voice and role control for full duplex conversational speech models. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (2026)
17. Saaty, T.L.: The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation. McGraw-Hill, New York (1980)
18. Thinking Machines Lab: Interaction models: a scalable approach to human–AI collaboration. https://thinkingmachines.ai/blog/interaction-models/ (2026). Accessed 29 July 2026
19. U.S. Congress: Sarbanes-Oxley Act of 2002. Public Law 107-204 (2002)
20. U.S. Department of Health and Human Services: Health Insurance Portability and Accountability Act of 1996. Public Law 104-191 (1996)
21. Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., Jégou, H.: The Faiss library. arXiv:2401.08281 (2024)
22. Kuchaiev, O., Li, J., Nguyen, H., Hrinchuk, O., Leary, R., Ginsburg, B., Kriman, S., Beliaev, S., Lavrukhin, V., Cook, J., et al.: NeMo: a toolkit for building AI applications using neural modules. arXiv:1909.09577 (2019)
23. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009)
24. Qwen Team: Qwen3 technical report. arXiv:2505.09388 (2025)

25. Nussbaum, Z., Morris, J.X., Duderstadt, B., Mulyar, A.: Nomic Embed: training a reproducible long context text embedder. arXiv:2402.01613 (2024)